DERS NOTU · *8 Kasım 2026*

Ders 23

NEXT.JS İLE FULL-STACK WEB GELİŞTİRME

Server Components ve Client Components

Bölüm 5 · App Router Temelleri

Kurs	*Bölüm*	*Tarih*
Next.js ile Full-Stack Web Geliştirme	Bölüm 5 - App Router Temelleri	8 Kasım 2026

Ders transkriptinden derlenmiştir · Kişisel kullanım içindir

İçindekiler

Bu notta yer alan başlıklar

Server Components ve Client Components 3

Özet 3

İki Ortam, İki Bileşen Türü 3

Server Components 3

Client Components ve "use client" 4

Sınırı Geçen Veriler: Props Kuralları 6

Kompozisyon Desenleri 6

Önemli Kavramlar 7

Dikkat Edilmesi Gerekenler 8

Tekrar Soruları 8

Server Components ve Client Components

Bu derste Next.js App Router'ın temelindeki ayrım ele alınıyor: bir bileşenin sunucuda mı yoksa tarayıcıda mı çalışacağı. Varsayılan olarak her bileşenin Server Component olduğu, `"use client"` yönergesinin bir sınır (boundary) çizdiği, iki tür arasında veri aktarımının kuralları ve istemciye gönderilen JavaScript miktarını küçük tutmak için bileşen ağacının nasıl kurgulanacağı örneklerle anlatılıyor.

1. İki Ortam, İki Bileşen Türü

Bir Next.js uygulamasında kod iki ortamda çalışabilir: isteği karşılayan **sunucu** ve sayfayı görüntüleyen **tarayıcı**. App Router, her bileşen için bu ortamlardan hangisinin sorumlu olduğunu açıkça belirlemenizi ister.

- **Server Component:** Yalnızca sunucuda çalışır. Kodu tarayıcıya hiç gönderilmez; sonucu (render çıktısı) gönderilir.
- **Client Component:** Etkileşim gerektiren bileşendir. İlk HTML yine sunucuda üretilir, ardından bileşenin JavaScript kodu tarayıcıya gönderilir ve sayfa "canlandırılır" (hydration).

`app/` klasöründeki her bileşen, aksi belirtilmedikçe **Server Component**'tir. Eğitmen bunun bilinçli bir tercih olduğunu vurguluyor: varsayılan durum "sıfır JavaScript"tir ve istemci kodu yalnızca gerçekten gerektiğinde eklenir.

EK NOT

"Client Component" adı yanıltıcı olabilir. Bu bileşenler de ilk yüklemede sunucuda HTML'e dönüştürülür; farkı, tarayıcıda yeniden çalışıp etkileşimli hale gelmeleridir.

2. Server Components

Doğrudan Veri Erişimi

Server Component'ler `async` olabilir. Bu sayede veriyi bileşenin içinde, `useEffect` veya ayrı bir API katmanı olmadan çekebilirsiniz:

```
// app/kurslar/page.tsx — Server Component (varsayılan)
import { db } from "@/lib/db";

export default async function KurslarSayfasi() {
  const kurslar = await db.kurs.findMany({ orderBy: { olusturulma: "desc" } });
```

```
  return (
    <ul>
      {kurslar.map((kurs) => (
        <li key={kurs.id}>{kurs.baslik}</li>
      ))}
    </ul>
  );
}
```

Veritabanı istemcisi, API anahtarları veya dosya sistemi gibi sunucuya ait kaynaklar burada güvenle kullanılabilir, çünkü bu kod tarayıcıya hiç ulaşmaz.

Sınırlamalar

Server Component'ler tarayıcıda çalışmadığı için şunları **kullanamaz**:

- `useState`, `useReducer`, `useEffect` gibi durum ve yaşam döngüsü hook'ları
- `onClick`, `onChange` gibi olay dinleyicileri
- `window`, `localStorage` gibi tarayıcı API'leri

3. *Client Components ve "use client"*

Bir dosyanın en üstüne `"use client"` yazmak, o modülü ve onun **içe aktardığı tüm modülleri** istemci paketine dahil eder. Yönerge bir bileşeni değil, modül ağacında bir **sınırı** işaretler.

DİKKAT

Bir Server Component içinde `useState` kullanmaya çalışmak derleme hatası verir. Hata mesajı genellikle bileşenin başına `"use client"` eklemenizi önerir; ancak önce o etkileşimin gerçekten bu bileşende olması gerekip gerekmediğini sorgulayın.

```tsx
// app/kurslar/begen-butonu.tsx
"use client";

import { useState } from "react";

export function BegenButonu({ baslangic }: { baslangic: number }) {
  const [sayi, setSayi] = useState(baslangic);

  return (
    <button onClick={() => setSayi((s) => s + 1)}>
      Beğen ({sayi})
    </button>
  );
}
```

Bu buton artık bir Server Component içinde normal şekilde kullanılabilir:

```tsx
// app/kurslar/[id]/page.tsx — Server Component
import { BegenButonu } from "../begen-butonu";
import { kursGetir } from "@/lib/kurslar";

export default async function KursDetay({ params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const kurs = await kursGetir(id);

  return (
    <article>
      <h1>{kurs.baslik}</h1>
      <p>{kurs.aciklama}</p>
      <BegenButonu baslangic={kurs.begeniSayisi} />
    </article>
  );
}
```

Sayfanın büyük kısmı sunucuda kalır; tarayıcıya yalnızca butonun kodu gönderilir.

4. *Sınırı Geçen Veriler: Props Kuralları*

Server Component'ten Client Component'e aktarılan props, ağ üzerinden taşınabilmesi için **serileştirilebilir** olmalıdır.

Aktarılabilir	Aktarılamaz
String, number, boolean, `null`	Sıradan fonksiyonlar (ör. `onSil={() ⇒ ...}`)
Düz nesneler ve diziler	Sınıf örnekleri (ör. veritabanı modeli nesnesi)
`Date`, `Map`, `Set`	Sembol (`Symbol`) değerleri
JSX (ör. `children`*)*	Veritabanı bağlantısı gibi sunucu kaynakları
Server Action fonksiyonları	

5. *Kompozisyon Desenleri*

Sınırı Aşağı İtmek

Eğitmenin en çok vurguladığı kural: `"use client"` yönergesini bileşen ağacında **mümkün olduğunca aşağıda**, yani yaprak bileşenlerde kullanın. Bütün bir sayfayı veya layout'u istemci bileşeni yapmak, içindeki her şeyi istemci paketine taşır.

Yaklaşım	Sonuç
`layout.tsx` *içine* `"use client"`	Tüm alt ağaç istemciye gider; sunucuda veri çekme avantajı kaybolur
Yalnızca arama kutusuna `"use client"`	Layout sunucuda kalır; tarayıcıya sadece arama kutusunun kodu gider

Server Component'i children Olarak Geçirmek

Bir Client Component, bir Server Component'i doğrudan `import` edemez; edilen modül istemci tarafına çekilir. Bunun yerine Server Component, ebeveyn tarafından `children` (veya başka bir JSX prop'u) olarak verilir:

```
// app/components/acilir-panel.tsx
"use client";

import { useState, type ReactNode } from "react";

export function AcilirPanel({ baslik, children }: { baslik: string; children:
```

NOT

Sunucudan istemciye fonksiyon aktarmanın tek yolu Server Action'dır: `"use server"` ile işaretlenmiş bir fonksiyon, istemciden çağrıldığında sunucuda çalışır. Konu bir sonraki derste ayrıntılı işleniyor.

```
  ReactNode }) {
    const [acik, setAcik] = useState(false);
    return (
      <section>
        <button onClick={() => setAcik(!acik)}>{baslik}</button>
        {acik && children}
      </section>
    );
  }
```

```
// app/kurslar/[id]/page.tsx
<AcilirPanel baslik="Müfredat">
  <Mufredat kursId={id} />   {/* Server Component: sunucuda render edilir */}
</AcilirPanel>
```

Burada `Mufredat` sunucuda render edilir ve sonucu panele hazır JSX olarak verilir. Panel yalnızca göster/gizle durumunu yönetir.

İPUCU

Bu desen, tema veya oturum bilgisi gibi React Context sağlayıcıları için de kullanılır: sağlayıcı ayrı bir Client Component dosyasında tanımlanır ve kök layout'ta `children`'ı sarar.

Sunucu Kodunu Korumak

Yalnızca sunucuda çalışması gereken bir modülü (ör. gizli anahtar kullanan bir yardımcı) yanlışlıkla istemciden içe aktarmayı önlemek için dosyanın başına `import "server-only";` eklenir. Böyle bir modül bir Client Component tarafından içe aktarılırsa derleme hatası alınır.

Önemli Kavramlar

Kavram	Açıklama
Server Component	Yalnızca sunucuda çalışan, kodu tarayıcıya gönderilmeyen bileşen; App Router'da varsayılan
Client Component	`"use client"` sınırının altında kalan, tarayıcıda etkileşimli hale gelen bileşen
`"use client"`	Modül ağacında sunucu–istemci sınırını işaretleyen yönerge
Hydration	Sunucuda üretilen HTML'e tarayıcıda olay dinleyicilerinin ve durumun bağlanması
Serileştirilebilir props	Sınırı geçebilen, ağ üzerinden taşınabilir veri türleri

Kavram	Açıklama
`server-only`	Bir modülün istemci paketine girmesini derleme anında engelleyen paket

Dikkat Edilmesi Gerekenler

> **DİKKAT**
>
> `"use client"` dosyanın **ilk satırı** olmalıdır; import'lardan sonra yazılırsa yok sayılır.

> **DİKKAT**
>
> Ortam değişkenlerinden yalnızca `NEXT_PUBLIC_` ön ekli olanlar istemci koduna gömülür. Gizli anahtarları bu ön ekle tanımlamayın ve yalnızca Server Component veya sunucu modüllerinde kullanın.

> **DİKKAT**
>
> Bir Client Component'e veritabanından gelen model nesnesini olduğu gibi geçirmek, gereksiz alanların (ör. e-posta, şifre özeti) tarayıcıya sızmasına yol açabilir. Yalnızca ihtiyaç duyulan alanları seçerek aktarın.

Tekrar Soruları

i. App Router'da bir bileşen, hiçbir yönerge yazılmadığında hangi türdedir ve bunun performansa etkisi nedir?

ii. `"use client"` neden tek bir bileşeni değil, bir modül sınırını işaretler?

iii. Bir Client Component içinde Server Component kullanmak için hangi desen uygulanır?

iv. Server Component'ten Client Component'e `onSil={() ⇒ ...}` şeklinde bir fonksiyon aktarılabilir mi? Neden?

v. `server-only` paketi hangi hatayı önler?